

Benchmark of AI Tools for Industrial Application Development

2D Palletization (Single-Bin, Python 3.10+, PyQt5)

Roman Paráček, JIC Smart Factory

April 30, 2026

1 Purpose of the Benchmark

The purpose of this benchmark is to objectively compare selected AI tools during the development of a realistic industrial application.

Each team member will implement an application that solves the problem of **2D palletization (2D bin packing)** on a single pallet, with the goal of using the pallet area efficiently. The application will include an algorithmic core and a graphical user interface built with PyQt5.

The benchmark will be carried out using the following tools:

- ChatGPT – Codex
- Cursor
- Claude Code

The outcome will be a functional application, a documented development process, and a structured evaluation of the AI tool.

1.1 Illustration of the 2D Palletization Problem

Figure 1 schematically illustrates the principle of 2D palletization (single-bin). The rectangle represents a pallet with dimensions $W \times H$. The smaller rectangles represent individual items. Items must not overlap and must respect the minimum distance g .

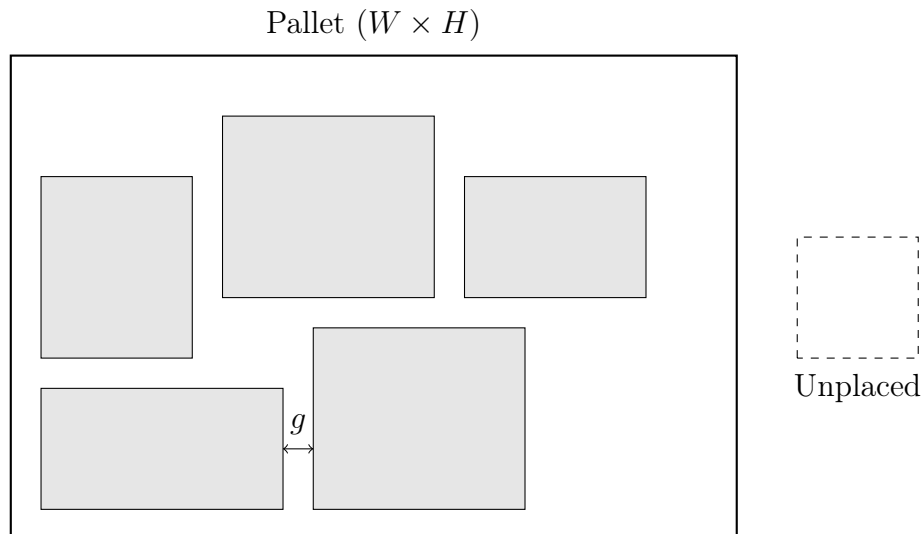


Figure 1: Schematic illustration of the 2D palletization problem (single-bin).

2 Functional Requirements

2.1 Purpose and Scope of the Solution

The task is to implement a solution for 2D palletization on a single pallet (single-bin).

The goal is to:

- prevent item overlap,
- maintain the minimum gap between items,
- ensure that items are placed inside the pallet,
- maximize pallet area utilization.

The solution may be heuristic. Finding the global optimum is not required.

2.2 Input Parameters

Pallet (Bin)

- W – pallet width [mm]
- H – pallet height [mm]
- g – minimum gap between two items [mm]

Constraints:

1. Each item must be placed inside the $W \times H$ area.
2. The minimum distance between any two items must be at least g .
3. No gap is required between an item and the pallet boundary.

Items

For item types $i = 1, \dots, n$:

- w_i – footprint width [mm]
- h_i – footprint height [mm]
- q_i – number of pieces of the given type (any non-negative integer)
- $rotation_i$ – permission to rotate by 90 degrees

It must be possible to define any number of pieces of the same item type. The application must also support a scenario with a single item type ($n = 1$), which corresponds to structured palletization.

If rotation is allowed, either orientation (w_i, h_i) or (h_i, w_i) may be used.

Input Validation

The application must:

- reject negative or zero item dimensions,
- reject inconsistent inputs (for example, an item larger than the pallet in both orientations),
- inform the user about the error through the GUI and the system logger,
- prevent computation from starting when the input data is invalid.

2.3 Optimization Objective

The optimization is defined lexicographically:

1. maximize the number of placed pieces,
2. among solutions with the same number of pieces, minimize unused area.

Unused area is defined as:

$$unused_area = W \cdot H - \sum_i (n_i^{placed} \cdot w_i \cdot h_i) \quad (1)$$

If not all items can be placed, the application must return the best valid solution and report the unplaced items (type and quantity).

2.4 Required Output

The application must provide:

- coordinates of each placed piece,
- its orientation,
- number of placed pieces,
- number of unplaced pieces,
- solution computation time,
- percentage area utilization.

3 Technical Requirements

- Language: Python 3.10+
- GUI framework: PyQt5
- Modular architecture (separation of GUI and logic)
- Type annotations and docstrings
- Public GitHub repository

3.1 GUI and Application Control Requirements

The graphical user interface (GUI) must be designed clearly and robustly, with a clear separation between the input section, the visualization area, and the diagnostic output.

1. Entering Input Parameters

The application must allow all input parameters (pallet and items) to be entered directly through the GUI.

The GUI must include:

- input fields for pallet dimensions (W , H),
- an input field for the minimum gap g ,
- the ability to define multiple item types (w_i , h_i , q_i , $rotation_i$),
- the ability to add and remove item types,
- support for the scenario with a single item type (structured palletization).

Alternatively, the application must allow input data to be loaded from a configuration file (for example, in JSON format). The configuration file format must be clearly described in the documentation (README).

Input validation must be performed when parameters are entered. If the data is invalid, the computation must not be allowed to start, and the user must be informed through the GUI and the system logger.

2. Controls

The GUI must include the following basic controls:

- **Run / Start** – starts the palletization computation for the current inputs,
- **Reset Scene** – clears the current visualization and returns the application to its initial

state,

- **Clear Logger** – clears the system log content.

Controls must be enabled only in the appropriate application state (for example, computation cannot be started with invalid data).

3. Visualization Area

The GUI must include a graphical area showing:

- the pallet ($W \times H$),
- individual placed items,
- their orientation,
- optional visual differentiation of unplaced items.

The visualization must be readable, scale-consistent, and must reflect the actual outputs of the algorithm.

4. System Logger

The application must include a system log (a separate panel in the GUI) that records application execution and supports diagnostics.

The logger must:

- display a timestamp and message level (INFO / WARN / ERROR),
- record key events (data loading, computation start, completion, metrics),
- record warnings (for example, partial solution, unplaced items),
- record errors and exceptions, including a brief description,
- support clearing its content using the **Clear Logger** button.

5. Error Handling and Stability

The GUI must be robust and must not crash during common error states. All errors must be caught and correctly reported to the user through a dialog or the logger.

After an error, the application must remain in a stable and reusable state.

4 Project Structure

The repository must have the following structure:

- App/ – application entry point (UI)
- src/ – application logic and algorithms
- Data/ – test instances and possible outputs
- Example/ – benchmark scripts
- images/ – images for the README
- Tests/ – automated tests

The repository must also include helper scripts to ensure a reproducible environment:

- **install.sh** / **install.bat** – automated installation script.

The script must:

- create a new Conda environment,
- install Python (minimum version 3.10),
- install all required libraries,
- allow the application to run fully without additional manual steps.

- **verify.py** – environment validation script.

The script must:

- verify the Python version,
- check the availability of all dependencies,
- test imports of the main application modules,
- confirm that the application can be started correctly.

The purpose of these scripts is to ensure a fully reproducible environment and smooth project startup on a clean system.

5 README.md Content

Each repository must include a professionally prepared `README.md` file in English. The README is the primary entry document for the project and must allow readers to quickly understand the purpose, architecture, and usage of the application without studying the source code.

The README must include, in particular:

- a clear and structured description of the project and its purpose,
- brief context for the 2D palletization problem,
- a visual example of the application (GUI screenshot),
- system requirements (Python version, dependencies),
- installation procedure (including use of `install.sh` / `install.bat` scripts),
- instructions for running the application,
- description of the project structure,
- overview of the implemented algorithms and their principles,
- known limitations and any assumptions of the solution.

The README must be written in technically precise and professional language and must correspond to the quality of typical open-source documentation.

6 Required Deliverables

Each benchmark participant is responsible for delivering the following outputs:

1. a public GitHub repository containing the complete source code,
2. a fully functional application that meets the defined requirements,
3. a README according to the specified requirements,
4. a brief professional reflection on the use of the AI tool (maximum 1 A4 page),
5. a 10–15 minute presentation of the results.

The reflection on the use of the AI tool should summarize the development experience and identify both the benefits and limitations of the tool in the context of industrial software development.

7 Evaluation Methodology

The evaluation is performed individually based on expert assessment. The purpose of the benchmark is not to compare participants with each other, but to provide a qualified assessment of individual AI tools.

During evaluation, it is recommended to focus especially on:

- the effectiveness of collaboration with the tool during development,
- the quality and maintainability of the resulting architecture,
- the amount of manual correction required compared with generated code,
- the tool’s ability to work with project context,

- the overall usability of the tool for industrial projects.

8 Benchmark Methodology

The benchmark is carried out as an individual implementation of the defined assignment using different AI tools. Each participant is responsible for the complete design, implementation, documentation, and presentation of the solution.

The purpose of the benchmark is not competition between participants, but the acquisition of structured experience with the practical use of individual AI tools.

9 Benchmark Objective

The objective of the benchmark is to obtain a qualified and structured overview of the capabilities of current AI tools in industrial software development.

The benchmark should provide answers to the following questions:

- To what extent are AI tools capable of independently designing and implementing a modular application of medium complexity?
- What is the quality of generated code in terms of architecture, readability, and extensibility?
- What is the actual amount of manual correction and intervention required?
- How well do the tools work with context across multiple files?

The benchmark should result in a qualified recommendation for the possible future deployment of AI tools within the development of industrial projects.